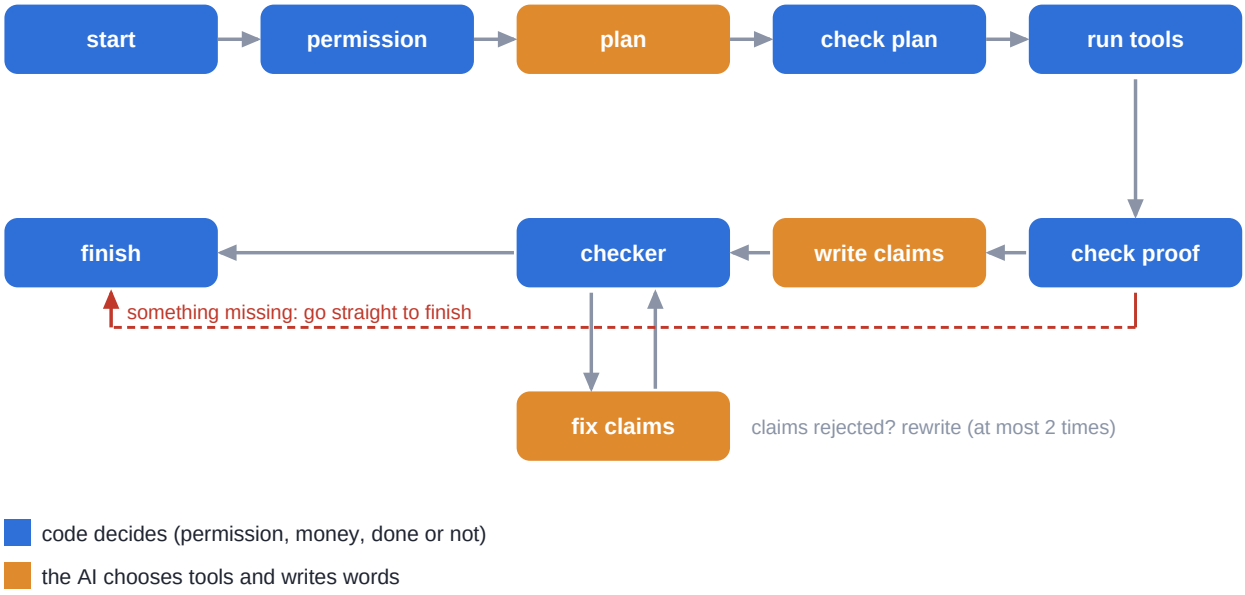
BLACKGRID

Sprint 2, in very simple English



How the helper learned to remember its work, ask before spending money, and recover after a crash. Short sentences. Every number is real. It comes from the project's own files.

1. The idea in one page

In Sprint 1 we built an honest computer helper. It studies who messaged whom, and it shows proof for everything it says.

But it had a weakness. If the computer crashed in the middle, the helper forgot everything. You had to start again. And if a paid AI question was in the air during the crash, nobody knew if it was paid for.

Sprint 2 fixes this. The helper now:

• saves its progress after every step,
• can stop, close, and continue later from the same place,
• never repeats work it already finished,
• never pays twice for the same question,
• asks you before it spends any money,
• shows you what it is doing at every moment.

The science did **not** change. Same data, same test, same rules. Only the way the helper works changed.

In short

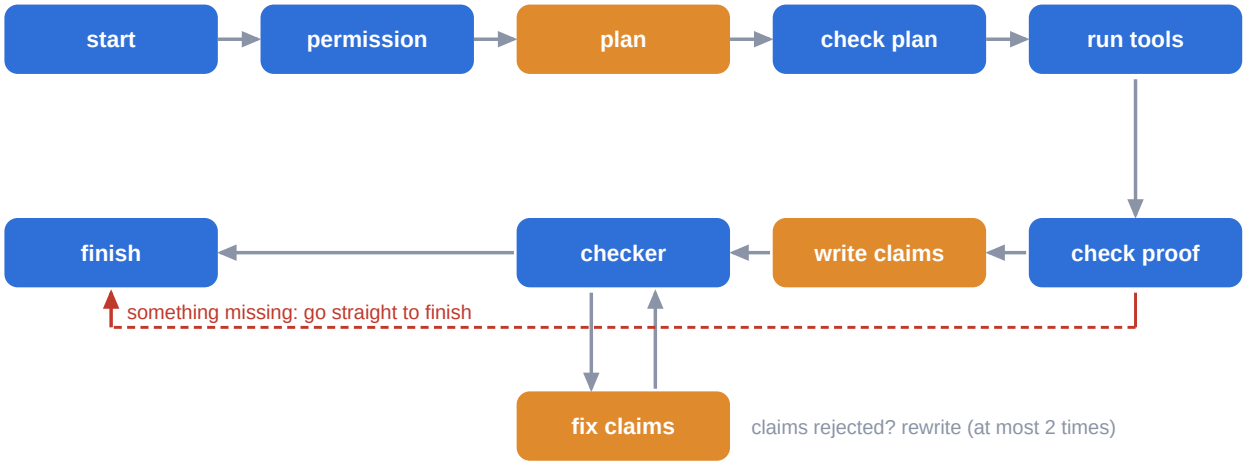
Sprint 1 made the helper honest. Sprint 2 makes it dependable: it remembers, it asks, and it recovers.

Success in one sentence

"The helper picks an allowed plan, runs everything the question needs, checks its own words, and can safely continue after an interruption. I can see every step and its cost."

2. The steps, made visible

Before, the helper's work was one big hidden block. Now it is a clear list of steps, like stations on a train line. The current station is always saved.



Step	Who decides	What happens
start	code	Load the data again and check it is the same data (its fingerprint).
permission	code + you	Only for paid runs. No permission? The helper stops and waits for you.
plan	AI	Suggests which allowed tools to use.
check plan	code	Refuses bad requests. Adds any test the rules need but the AI forgot.
run tools	code	Runs the tests one by one. Saves each result.
check proof	code	Every needed test must exist and be unchanged. If not, stop as "partial".
write claims	AI	Writes short sentences, each pointing to its proof.
checker	code	Checks each sentence with 7 fixed rules.
fix claims	AI	Rewrites rejected sentences. At most 2 tries.
finish	code	Decides the result. Writes the report and the web page.

In short

The AI chooses tools and writes words. **Code** decides permission, money, and whether the job is done. The AI can never overrule the code.

This step list is built with a tool called **LangGraph**. It is a free library for building step lists that can save and continue. We use version 1.2.12.

3. Saving progress

After every step, the helper writes down where it is. This is called a **checkpoint**. Think of it like a save point in a video game.

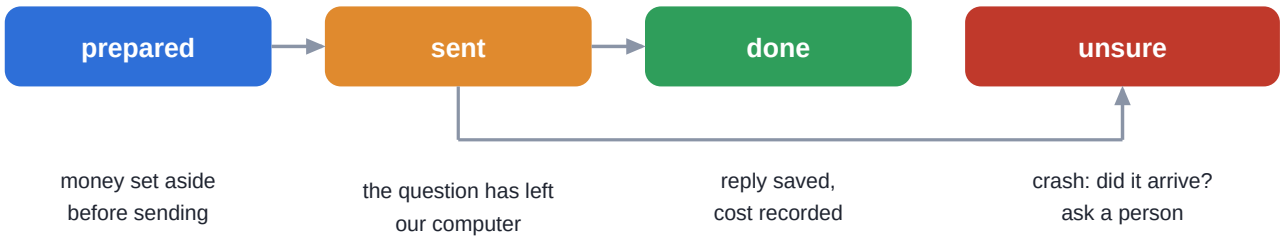
The saves go into two small database files on your computer:

• **checkpoints.sqlite**: which step the run is on.
• **workflow.sqlite**: every AI question and every tool run, and what each one cost.

What is **never** saved there: your API key, the raw messages, people's IDs, or the hidden right answers of the practice tests. A test checks this.

Each AI question has its own record

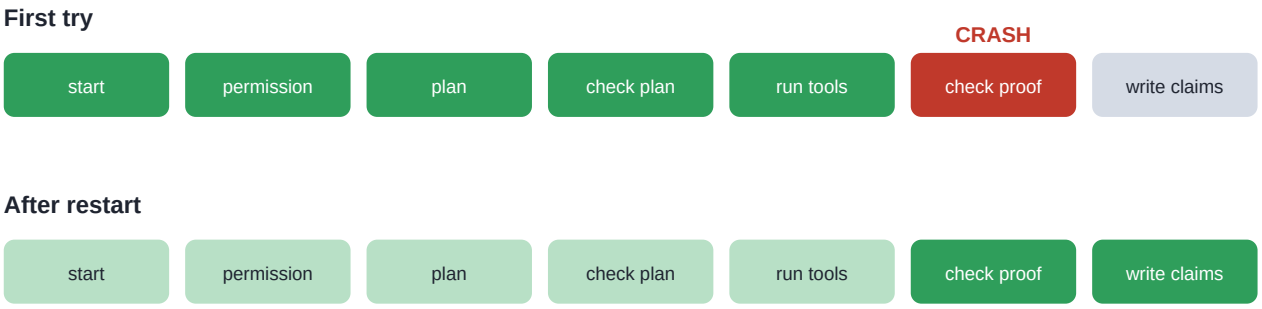
Before an AI question leaves the computer, the helper writes a record first. The record moves like this:



Record says	What it means	On restart
prepared	Money is set aside. The question has not been sent.	Safe to send now.
sent	The question may have reached the AI.	If the reply was saved, use it. If not, ask you.
done	Reply saved. Cost recorded.	Use the saved reply. Never ask again.
failed	A known problem, like a broken reply.	Try once more, if the limits allow.
unsure	Nobody knows if it arrived or was charged.	Keep the full cost counted. Ask you.

4. What happens after a crash

Say the computer shuts down in the middle of a run.



Light green = reused from the saved record (not run again). Dark green = runs now.

When you start it again, the helper reads its saves. Steps that finished are **reused**. It does not run them again. It does not pay for them again. It just continues.

You restart with one command:

```
uv run --frozen blackgrid resume RUN_ID --force-unlock
```

Why **--force-unlock**? The crashed program never said "I'm done". So the run still looks busy. This flag says "that program is really gone". Without it, a second copy is not allowed to touch the same run. That stops two copies from fighting.

The one case nobody can decide

What if the crash happened **after** a question was sent, but **before** the reply was saved? Then nobody can know if the AI answered and charged for it. Not even us.

So the helper does **not** guess and does **not** send it again. It stops and asks you:

```
uv run --frozen blackgrid reconcile RUN_ID --retry
```

--retry tries once more, inside the limits. **--stop** ends the run. Either way, the unsure question stays counted at its full price, to be safe.

Be careful

No system can promise that a paid question is sent exactly once. A crash at the wrong moment is always unclear. BLACKGRID is honest about this: it stops and asks instead of guessing.

5. Money limits that survive restarts

Every time, the helper adds up the money, calls, and words from the saved records. It never keeps the count only in memory. So a restart can never reset the count to zero.

Limit	Amount
Per run	8 AI calls, 20,000 words (tokens), $0.50
Sprint 2 in total	$1.00
Whole project	$10.00 (this includes the $0.0051 spent in Sprint 1)
Retries	1 retry per failed call
Rewrite rounds	2

Before every paid question, the helper reserves the **worst possible cost**. If that could break any limit, the question is not sent.

Errors are sorted into kinds, so the helper knows what to do:

Kind of error	Example	What the helper does
temporary	No internet; the AI service is busy	Try once more
broken reply	The reply was cut off	Try once more, and ask for a shorter reply
permanent	Wrong model name, wrong key	Do not try again
unsure	Timed out after sending	Stop and ask you

In short

Money is counted from saved records, limits are checked before each call, and an unclear charge is always counted in full.

6. Asking before spending

A **mock** run uses a free, fake AI. It never needs permission.

A **live** run uses a real AI and costs money. Now, if you start it without permission, the helper stops at the "permission" step and waits. Nothing is spent. Even the planner is paid, so it waits before that too.

You want to	Command
start and pause for permission	`blackgrid run configs/collegemsg_live.json`
give permission	`blackgrid resume RUN_ID --allow-paid`
say no	`blackgrid deny RUN_ID`
stop a run	`blackgrid cancel RUN_ID`
see where a run is	`blackgrid status RUN_ID`

Permission covers only **this** run: this model, this data, these rules, these tools, this cost limit. It lasts 24 hours. If you change the model or the data, the permission no longer fits, and the helper asks again.

We tested this without spending money

We started a live run without permission. It stopped at "permission" with all 8 calls and all $0.50 still unused. Then we said no. The result was "denied", with 0 calls and $0 spent.

All possible results

Result	Simple meaning
completed	Everything needed was done and checked.
partial	It finished, but a needed test is missing. Only part of the question is answered.
abstained	The data broke a science rule written in advance, so no answer.
failed	A tool, the AI service, or a safety check failed.
budget used up	The next call would break a money or size limit.
paused	Waiting for you: permission, or an unsure charge.
denied	You said no to spending.
cancelled	You stopped it.

7. The new dashboard

You asked for a live, interactive dashboard. Now there is one:

```
uv run --frozen blackgrid console
```

Then open **http://127.0.0.1:8765** in your web browser.

• It shows every run and updates by itself while a run is working.
• A bar of steps shows which step the run is on.
• Tabs show the test, the network drawing, the missing-data test, the helper's steps, the sentences, the report, and the money.
• You can hover over charts, zoom the network, and sort tables.
• Buttons let you start a free run, run the practice comparison, and resume, cancel, deny, or settle a run.

Be careful

The dashboard can **never** start a paid run or give permission to spend. That is only possible from the command line, on purpose.

Safety for the dashboard

• It only works on your own computer (address 127.0.0.1).
• Every button needs a secret code that only the real page has.
• Other websites cannot reach it or put it inside their pages.
• Text from the AI is shown as plain text, so it cannot run as code.

A saved copy of every run

When a run finishes, the same dashboard is saved as one file in the run's folder: **report.html**. It works without the server and without the internet. Just double-click it. **runs/index.html** lists all runs.

8. How we tested it

We did not trust it until we tried to break it. We made the program "crash" on purpose at chosen moments, then started it again like a brand-new program.

We crashed it	What we checked	Result
after a test finished	the test is not run again	passed
just before sending a question	it sends it once, not twice	passed
just after sending a question	it stops and asks you; it does not resend	passed
after the reply was saved	it uses the saved reply	passed
while writing the report	it finishes the report; no extra AI calls	passed

We also checked that:

- limits still work after a restart,
- two copies cannot run the same job at once,
- old, expired, or wrong permission makes it wait again,
- a changed result file is caught and the run fails,
- a changed setting blocks continuing,
- the saves hold no secret key and no hidden answers,
- a replay never secretly calls a real AI,
- a missing needed test gives "partial", never "completed".

9. Same science, same answer

Changing how the helper works must not change the science. So we compared the new version with Sprint 1.

• The real data run gave **exactly** the same numbers: group score 0.587, chance average 0.593, and 358 of 499 shuffled copies at least as high. Same answer: "not supported".
• We did it again in a **brand-new, empty folder** with no internet and no saved results. Again exactly the same. 171 tests passed there (1 was skipped because that folder has no raw data, on purpose). It took 3.3 minutes.

• In the practice comparison, **15 of 16** runs were exactly the same.

The one different run was a change we wanted. A careless helper asked to run its own code. That was refused, so no test ran at all. Before, this was called "abstained". Now it is called "failed", which is more honest. And the helper no longer wastes an AI call writing about missing results.

In short

The new way of working gives the same science. Nothing hidden changed.

10. Problems we found and fixed

Problem	Fix
A rewrite round was counted even when no call was made.	Count a round only if its call happened.
The web page showed an old step after the run finished.	Save the final result first, then draw the page.
The page crashed for a run you said no to (no price info).	Handle missing price info.
The new dashboard showed some boxes as text.	Fix how the page builds its parts.
On Windows, a refused button press sometimes looked like a network error.	The server now reads the full request before it answers.
A crash while fixing sentences could ask the AI twice.	Each round now has a fixed number.
An edited saved AI reply was trusted.	Every saved reply is fingerprint-checked before use.
Running out of time could skip writing the result.	The finish step always runs.
The dashboard blocked Cancel while another job ran.	Cancel is always allowed.

Every fix got its own test, so the problem cannot quietly come back.

11. Time and money

After the tests, you ran **one real-AI run** with the new safety features. It **completed**. Both needed tests ran. The real AI wrote 4 sentences, and the checker accepted all 4 on the first try. Every charge stayed below the money set aside for it. The science was exactly the same as before.

	Amount
Money spent in Sprint 2	$0.0016 (one real-AI run, 2 calls)
Money spent in the whole project	$0.0067 ($0.0051 in Sprint 1 + $0.0016 in Sprint 2)
Sprint 2 money limit	$1.00
Computer working time	about 1 hour
Human hours	not measured, so not claimed

12. Honest limits

• A paid question can never be promised to go out exactly once. Unclear cases stop and ask.
• Cancel is polite: it waits for the current step to finish.
• The database files are not a security wall. There is still no full computer sandbox.
• The dashboard is for one person on one computer. It has no user accounts.
• Working more reliably does **not** mean the science is more correct.
• We still have not shown that the checker helps a **real** AI. The practice comparison uses a fake, scripted helper.

13. How to run it

Set up (once)

```
uv sync --frozen
```

Run all tests

```
uv run --frozen pytest
```

Open the live dashboard

```
uv run --frozen blackgrid console
```

Free run on the real data

```
uv run --frozen blackgrid run configs/collegemsg_sprint01.json
```

See all runs

```
uv run --frozen blackgrid status
```

Continue a run

```
uv run --frozen blackgrid resume RUN_ID
```

Real AI run (costs a little)

```
uv run --frozen blackgrid run configs/collegemsg_live.json --allow-paid
```

A plain-English guide with more detail is in **docs/workflow_guide.md**.

14. Hard words, made simple

Word	Simple meaning
Workflow	The fixed list of steps the helper follows
Step / node	One station on that list
Checkpoint	A save point: where the run is
Resume	Continue from the last save point
Durable	Saved on disk, so it survives a crash
Operation record	A note written before each AI call or tool run
Reserve	Set money aside before a call, at the worst possible cost
Reconcile	A person decides what to do with an unclear charge
Authorization	Your permission to spend money
Scope	Exactly what a permission covers
Lock	A sign that says "this run is busy"
Supervisor	The code that decides permission, money, and when the job is done
LangGraph	The free library we use to build the step list
SQLite	A small database stored as one file
Parity	Proof that the new version gives the same results as the old one
Snapshot	A saved copy of the dashboard that works offline
Console / dashboard	The web page that shows all runs
Mock	A fake, scripted AI for free testing

15. What comes next

1. Done: one small real-AI run with the new safety features ($0.0016).
2. One controlled crash-and-recover test with the real AI.
3. Sprint 3: new practice cases and a fair, independent test of whether the checker really helps.
4. Later, maybe: compare the Louvain method with a newer one called Leiden, with its own rules written first.